

Архитектурное описание системы

1. Общие сведения о системе

Разрабатываемая система представляет собой однопользовательскую игру в жанре roguelike с консольным интерфейсом. Игрок управляет персонажем с клавиатуры, перемещается по карте уровня, взаимодействует с мобами и предметами. Предметы могут находиться на карте, подбираться в инвентарь, а также надеваться и сниматься, надетые предметы влияют на характеристики персонажа. Игра состоит из последовательности уровней, уровни могут быть заранее определёнными (загружаться из набора уровней) или генерироваться процедурно. Игра завершается победой при прохождении всех уровней либо поражением при смерти персонажа.

2. Architectural drivers

При проектировании системы учитываются следующие факторы. Архитектура должна быть расширяемой, так как дальнейшие задания предполагают развитие механик и усложнение логики. Требуется высокая скорость разработки, компоненты должны быть слабо связаны, чтобы изменения в генерации уровня, данных или рендере не приводили к переработке всего приложения. Интерфейс должен работать без заметных задержек при вводе и перерисовке. Дополнительным ограничением является простота реализации и понятность структуры для учебного проекта.

3. Роли и случаи использования

В системе присутствует один основной актор - игрок. Игрок запускает приложение и взаимодействует с игрой через клавиатуру. Система отображает состояние уровня и результаты действий пользователя.

3.1. Прецедент П1. Игра в Roguelike игру

Основной исполнитель: игрок.

Заинтересованные лица и их требования: игрок хочет получить удовольствие от игры и проникнуться духом старых игр.

Основной успешный сценарий.

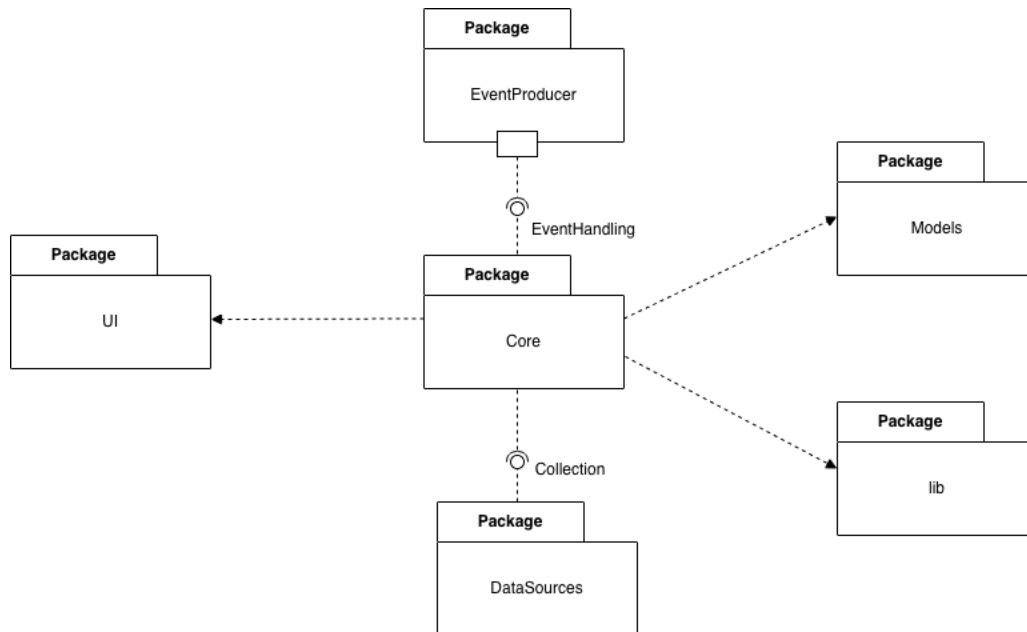
1. Игрок запускает игру.
 2. Система предлагает пользователю ввести имя персонажа, после чего пользователь вводит имя.
 3. Далее система предлагает пользователю выбрать пол, расу и класс персонажа - пользователь делает выбор.
 4. После завершения ввода параметров персонажа система генерирует новый уровень.
 5. Пользователь проходит уровень, уничтожая всех мобов.
 6. После выполнения условия завершения уровня система предлагает перейти на новый уровень.
 7. Пользователь подтверждает переход.
- Действия, описанные в пп. 5-7, повторяются, пока пользователь не пройдет все уровни.
8. После прохождения последнего уровня система поздравляет пользователя с прохождением игры.

Расширения: при закрытии пользователем терминала с игрой приложение завершается без выполнения дополнительных действий.

4. Диаграмма компонентов и описание компонентов

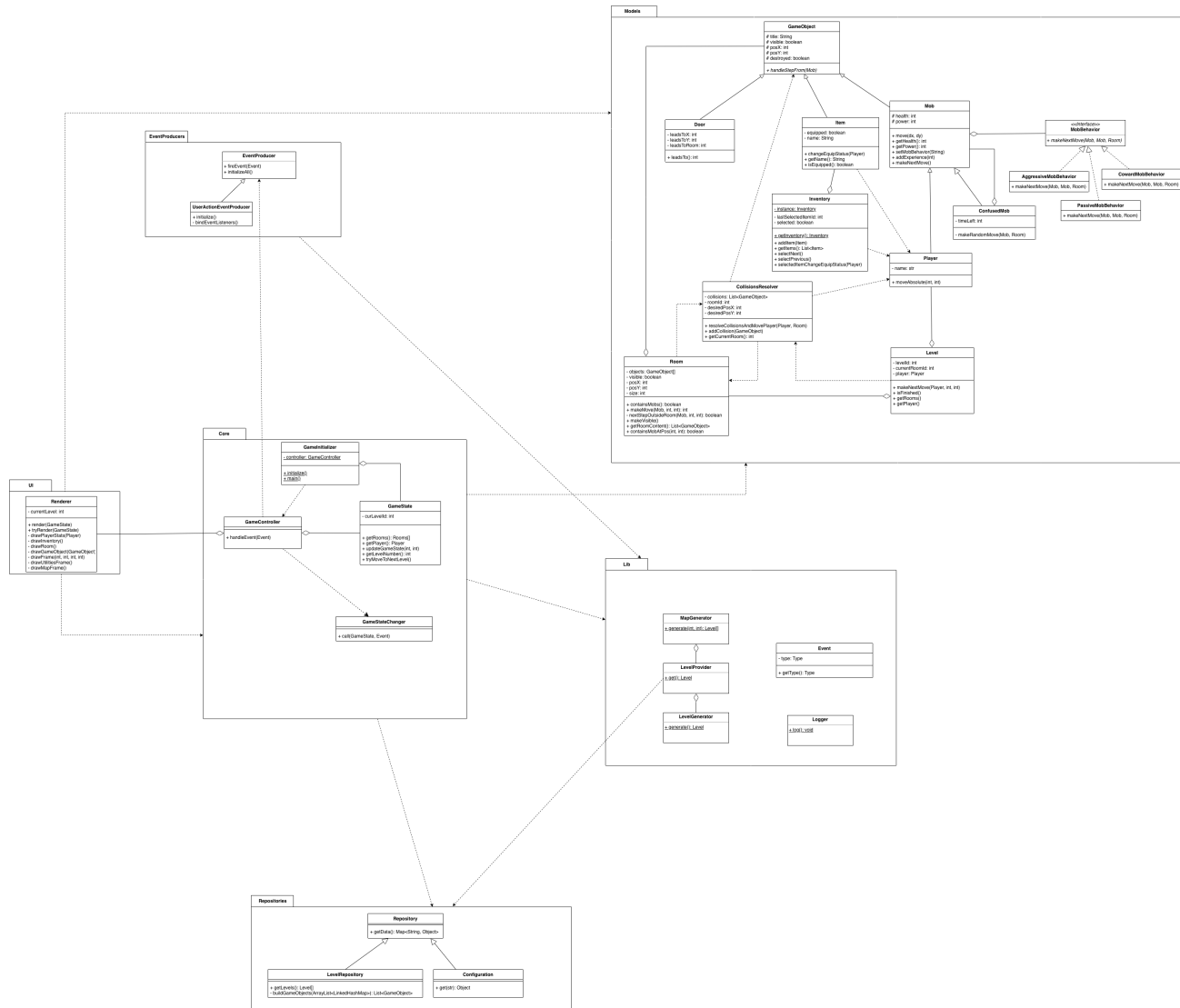
Система разделена на набор основных компонентов. Компонент генерации событий ответственен за формирование событий, на которые система должна реагировать. Например, он может слушать нажатия клавиш и создавать соответствующие события, либо генерировать события с заданной периодичностью.

Компонент ядра ответственен за обработку событий, обновление состояния игры и вызов перерисовки интерфейса. Интерфейс занимается отрисовкой текущего состояния системы и инкапсулирует выбранную UI-библиотеку.



Компонент доменной модели представляет набор сущностей предметной области, используемых другими компонентами. Отдельно выделены вспомогательные алгоритмы (например, генерация уровней) и репозитории, содержащие конфигурируемые параметры игры (набор уровней, параметры генерации и прочие данные).

5. Диаграмма классов и логическая структура



Подсистема генерации событий отвечает за создание событий при действиях пользователя и передачу их в ядро для обработки.

Классы доменной модели используются для хранения информации об игровых объектах. Игровой объект может быть вещью, мобом (игрок или программно управляемый противник) либо переходом между комнатами на одном уровне. Каждый игровой объект обладает названием и координатами относительно уровня и определяет поведение при взаимодействии с мобом (например, при наступании на клетку).

Поведение врагов реализовано с помощью паттерна «Стратегия»: выделен интерфейс поведения моба и несколько реализаций (например, агрессивное, пассивное и трусливое). Такой подход позволяет добавлять новые варианты поведения без изменения класса моба.

Дополнительно предусмотрена механика временного эффекта (например, оглушение). Она реализована через паттерн «Декоратор» - обертка над мобом изменяет его логику движения на период действия эффекта.

Уровень состоит из комнат, которые содержат игровые объекты. Переходы между комнатами реализуются отдельными объектами перехода.

В ядре располагается обработка событий и изменение состояния игры: при получении события обновляется текущее состояние, после чего обновляется пользовательский интерфейс. Состояние игры агрегирует текущий уровень и состояние игрока.

Во вспомогательных модулях располагаются реализации алгоритмов, необходимых для работы ядра, и утилиты, такие как генераторы уровней/карты и логирование.

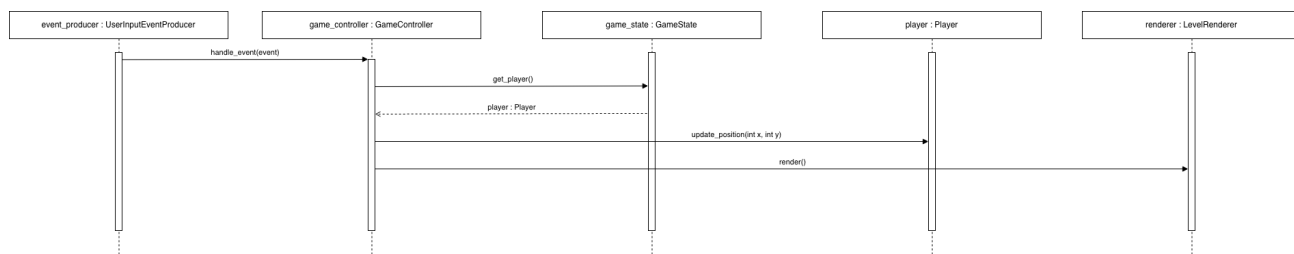
Репозитории содержат данные, необходимые остальным компонентам: набор предопределенных уровней и конфигурируемые параметры игры.

Интерфейс отвечает за отображение уровня и служебных элементов. Инициализацией игры занимается отдельный компонент, который подготавливает все необходимые объекты и запускает основной цикл.

6. Диаграммы последовательностей (взаимодействия)

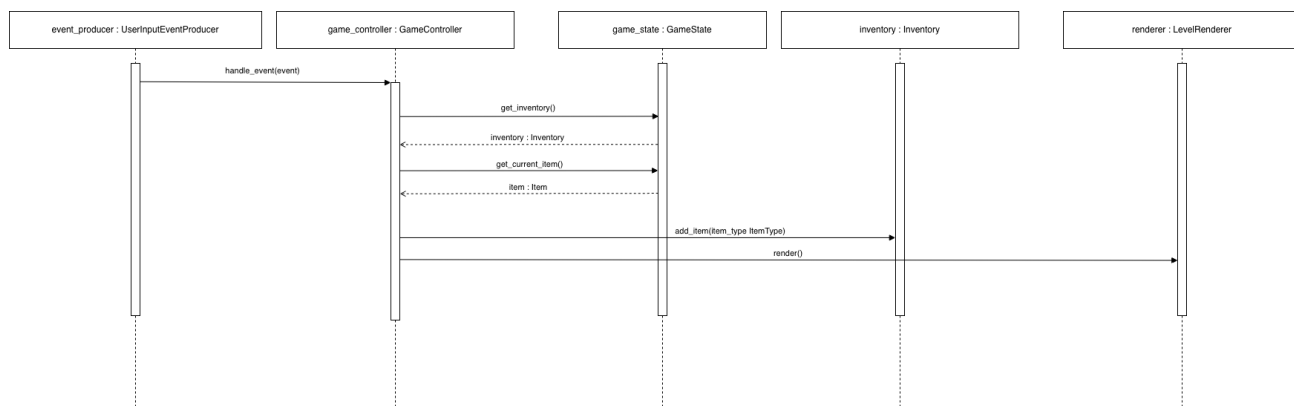
Диаграммы последовательностей фиксируют типовые сценарии работы системы и показывают передачу управления между объектами при обработке событий пользователя.

6.1. Перемещение персонажа



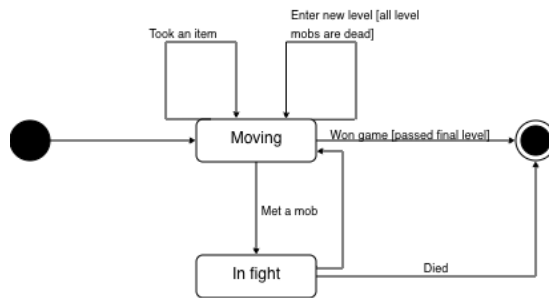
`UserActionEventProducer` формирует событие и инициирует его обработку в `GameController.handle_event(Event)`. `GameController` получает игрока через `GameState.get_player(): Player` и изменяет позицию игрока (например, через метод `Player.update_position(int x, int y)` или эквивалентный вызов). После изменения состояния `GameController` вызывает `Renderer.render(GameState)`.

6.2. Подбор предмета



`UserActionEventProducer` формирует событие подбора предмета и передаёт его в `GameController.handle_event(Event)`. `GameController` получает доступ к `Inventory` (через `GameState.inventory` или через аксессор) и добавляет предмет в инвентарь (например, вызовом `Inventory.add_item(...)`). Далее выполняется `Renderer.render(GameState)`.

7. Диаграмма состояний



Состояния игры представлены режимами Moving (исследование уровня), In fight (бой) и Game over (завершение игры). Переход Moving > In fight происходит при встрече с мобом. После боя возможен переход в Moving при победе или переход в Game over при смерти игрока. Переход на следующий уровень выполняется при выполнении условия завершения уровня и не меняет основной режим (остается Moving), но обновляет текущий Level. При прохождении последнего уровня выполняется переход в Game over с результатом “победа”.